

# PT07 - Generic

## Petunjuk

1. Perintah untuk mengumpulkan berkas kode program dan jawaban akan dituliskan dalam warna **merah**.
2. Pertanyaan yang harus Anda jawab akan dituliskan dalam warna **biru muda** dan memiliki nomor. **Tulis jawaban anda pada sebuah berkas teks dengan nama PT0701xxyyy.txt dengan xx adalah tahun angkatan dan yyy adalah nomor urut mahasiswa.**

## 1. Tipe Generik

Pada praktikum kali ini, kita akan belajar fitur *generic* pada bahasa Java. Sebagai studi kasus, kita akan membuat kelas “Kardus”, yang fungsinya menyimpan berbagai macam objek di dalamnya.

### Kelas Kardus

Perhatikan kode kelas kardus di bawah ini:

```
public class Kardus {  
    private Object isi;  
    public Kardus(Object isi) {  
        this.set(isi);  
    }  
    public void set(Object isi) {  
        this.isi = isi;  
    }  
    public Object get() {  
        return this.isi;  
    }  
}
```

Kelas ini fungsinya untuk menyimpan berbagai jenis objek yang kita inginkan. Sebagai contoh, kita buat Tester di bawah ini<sup>1</sup>:

```
public class Tester {  
    public static void main(String[] args) {  
        Kardus stringKardus = new Kardus("Hello");  
        Kardus doubleKardus = new Kardus(Double.valueOf(3.14));  
        System.out.println(stringKardus.get());  
        System.out.println(doubleKardus.get());  
    }  
}
```

1. Jika konstruktor kelas Kardus mengharapkan tipe data “Object”, mengapa dalam kasus ini bisa diisi dengan String ataupun Double pada Tester?

2. Coba tambahkan satu baris pada Tester seperti di bawah ini. Apakah program tersebut masih bisa dikompilasi? Pesan kesalahan apa yang muncul, dan apa maksudnya?

```
System.out.println(doubleKardus.get().doubleValue());
```

---

<sup>1</sup> Fungsi “Double.valueOf()” berfungsi untuk mengonversi primitif double menjadi kelas Double.

## Versi Generik dari Kardus

Sebuah kelas generik didefinisikan dengan format berikut:

```
class name<T1, T2, ..., Tn> { /* ... */ }
```

T1, T2, ... Tn yang mengikuti nama kelas menspesifikasikan *type parameters* (dikenal juga dengan *type variables*).

Berikut adalah kelas Kardus yang sudah dimodifikasi menjadi generik. Silakan ubah kelas Anda menjadi seperti di bawah ini. Perhatikan bahwa selain ada penambahan spesifikasi *type parameter* T, seluruh referensi Object kita ubah menjadi T.

```
public class Kardus<T> {  
    private T isi;  
    public Kardus(T isi) {  
        this.set(isi);  
    }  
    public void set(T isi) {  
        this.isi = isi;  
    }  
    public T get() {  
        return this.isi;  
    }  
}
```

Ubah pula kelas Tester sehingga menjadi seperti di bawah ini. Perhatikan pula bahwa saya juga menyertakan baris penyebab error tadi. Cobalah kompilasi dan jalankan, dan kali ini tidak akan error lagi.

```
public class Tester {  
    public static void main(String[] args) {  
        Kardus<String> stringKardus = new Kardus<>("Hello");  
        Kardus<Double> doubleKardus =  
            new Kardus<>(Double.valueOf(3.14));  
        System.out.println(stringKardus.get());  
        System.out.println(doubleKardus.get());  
        System.out.println(doubleKardus.get().doubleValue());  
    }  
}
```

Seperti Anda lihat, pemanfaatan kelas sebagai tipe data kali ini mengandung parameter kelas lain, antara String atau Double. Pemanggilan konstruktor juga berubah, menggunakan *diomond* "<>" yang artinya secara otomatis menyesuaikan dengan kelasnya.

Dengan menggunakan *type parameter* ini, secara otomatis sudah diketahui saat *compile time* bahwa tipe kembalian dari Kardus<String>.get() adalah String, dan tipe kembalian dari Kardus<Double>.get() adalah Double, menyesuaikan parameternya.

3. Mengapa pemanggilan doubleValue() kali ini tidak menimbulkan error seperti sebelumnya? (tip: lihat kembali pesan kesalahan yang muncul sebelumnya, dan perhatikan tipe kembaliannya sekarang)

## Konvensi Penamaan

Huruf “T” yang kita gunakan bisa diganti dengan apapun. Namun, Oracle menyarankan konvensi penamaan seperti berikut:

- E - *Element* (banyak digunakan pada *Java Collections Framework*)
- K - *Key* (kunci)
- V - *Value* (nilai)
- N - *Number* (bilangan)
- T - *Type* (tipe)
- S,U,V etc. - tipe ke-2, 3, dst jika dibutuhkan lebih dari 1 tipe

## Multiple Type Parameters & Inheritance

Ketik dan perhatikan contoh kode berikut.

```
public interface Pasangan<K,V> {  
    public K getKey();  
    public V getValue();  
}
```

```
public class PasanganKonkrit<K,V> implements Pasangan<K,V> {  
    private K key;  
    private V value;  
  
    public PasanganKonkrit(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public K getKey() {  
        return this.key;  
    }  
  
    public V getValue() {  
        return this.value;  
    }  
}
```

Perhatikan bahwa kode di atas mendemonstrasikan:

1. Tipe Generik bisa digunakan juga pada sebuah interface
2. Dalam satu tipe, bisa menggunakan lebih dari 1 template
3. Tipe Generik bisa diturunkan

Ujilah kode di atas dengan tester berikut<sup>2</sup>:

---

<sup>2</sup> Backup terlebih dulu file Kardus.java Anda sebelum mengetik kode ini, karena akan digunakan kembali di bagian berikutnya.

```

public class Tester {
    public static void main(String[] args) {
        Pasangan<String, String> identitasMahasiswa =
            new PasanganKonkrit<>("6182201002", "Fernando");
        Pasangan<String, Double> ipkMahasiswa =
            new PasanganKonkrit<>("6182201002", 3.98);
        System.out.println(identitasMahasiswa);
        System.out.println(ipkMahasiswa);
    }
}

```

Perhatikan pula bahwa:

1. Konstruktor instansiasi berbeda dengan (merupakan turunan dari) tipe datanya. Ini menunjukkan bahwa *inheritance* berfungsi seperti biasa pada *generic type*.
2. Kita langsung menuliskan "3.98", bukan "Double.valueOf(3.98)". Ini berkat fitur autoboxing<sup>3</sup> yang otomatis dari Java mengonversi primitif ke versi kelas nya.

4. Implementasikan method `toString()` pada kelas `PasanganKonkrit` yang menampilkan nilai key dan valuenya. Pastikan file `PasanganKonkrit.java` yang Anda kumpulkan nanti mengandung implementasi ini.

## 2. Method Generik

Method generik adalah method yang memiliki tipe parameter. Cara penggunaannya mirip dengan tipe generik.

Sebagai contoh, cobalah tambahkan method static pada `Tester` Anda seperti di bawah ini<sup>4</sup>.

```

public class Tester {
    public static <K,V> boolean sama(Pasangan<K,V> p1, Pasangan<K,V> p2) {
        return p1.getKey().equals(p2.getKey()) &&
            p1.getValue().equals(p2.getValue());
    }

    // ...
}

```

Method `sama` tersebut mengambil dua buah parameter dan memeriksa apakah isinya sama atau tidak. `<K,V>` sebelum tipe kembalian mendeklarasikan template yang ingin dipakai, dan simbol `K` dan `V` dapat digunakan dimanapun di deklarasi parameter maupun isi dari method. Pada contoh di atas, `K` dan `V` digunakan pada tipe parameter.

Untuk menggunakan method tersebut, silakan ubah method `main` Anda seperti berikut:

<sup>3</sup> <https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html>

<sup>4</sup> Method generik juga bisa digunakan pada method non-static.

```

public class Tester {

    // ...

    public static void main(String[] args) {
        Pasangan<String, String> identitasMahasiswa =
            new PasanganKonkrit<>("6182201002", "Fernando");
        Pasangan<String, Double> ipkMahasiswa =
            new PasanganKonkrit<>("6182201002", 3.98);
        System.out.println(identitasMahasiswa);
        System.out.println(ipkMahasiswa);
        Pasangan<String, String> identitasMahasiswa2 =
            new PasanganKonkrit<>("6182201002", "Fernando");
        System.out.println(Tester.<String,String>sama(
            identitasMahasiswa, identitasMahasiswa2));
    }
}

```

Coba compile dan jalankan programnya, dan pastikan Anda mendapatkan hasil “true” dari kembalian fungsi sama().

5. Cobalah untuk menghapus “<String,String>” dari perintah

“System.out.println(Tester.<String,String>sama(identitasMahasiswa, identitasMahasiswa2));”.

Apakah program tetap dapat dikompilasi? Mengapa demikian?

### 3. Bounded Type Parameters

Kita dapat membatasi tipe data yang digunakan dalam template. Sebagai contoh, kita tambahkan “extends Number” pada template kelas Kardus di bawah ini:

```

public class Kardus<T extends Number> {
    private T isi;
    public Kardus(T isi) {
        this.set(isi);
    }
    public void set(T isi) {
        this.isi = isi;
    }
    public T get() {
        return this.isi;
    }
}

```

Dengan demikian, saat digunakan T hanya bisa diisi dengan kelas Number dan turunannya (Double, Integer, dll).

Coba jalankan kembali kelas Tester yang sebelumnya pernah dibuat:

```

public class Tester {
    public static void main(String[] args) {
        Kardus<String> stringKardus = new Kardus<>("Hello");
        Kardus<Double> doubleKardus =
            new Kardus<>(Double.valueOf(3.14));
        System.out.println(stringKardus.get());
        System.out.println(doubleKardus.get());
        System.out.println(doubleKardus.get().doubleValue());
    }
}

```

6. Pesan kesalahan apa yang Anda dapatkan? Jelaskan artinya!

7. Variabel apa yang perlu Anda hilangkan dari method main() tersebut? *Comment out*<sup>5</sup> juga pada kode program Anda sehingga dapat dikompilasi dan dijalankan.

Jika kita membatasi tipe data template ke sebuah kelas, kita dapat memanfaatkan method-method yang dimiliki kelas tersebut. Sebagai contoh, kelas Number memiliki method “doubleValue()” dan kita bisa menggunakannya di dalam kelas Kardus...

```
public class Kardus<T extends Number> {
    private T isi;
    public Kardus(T isi) {
        this.set(isi);
    }
    public void set(T isi) {
        this.isi = isi;
    }
    public T get() {
        return this.isi;
    }
    public double getDoubleValue() {
        return this.isi.doubleValue();
    }
}
```

... dan bisa dimanfaatkan pula di dalam tester:

```
public class Tester {
    public static void main(String[] args) {
        // Kardus<String> stringKardus = new Kardus<>("Hello");
        Kardus<Double> doubleKardus = new Kardus<>(3.14);
        // System.out.println(stringKardus.get());
        System.out.println(doubleKardus.get());
        System.out.println(doubleKardus.get().doubleValue());
        System.out.println(doubleKardus.getDoubleValue());
    }
}
```

Ubahlah kode Anda menjadi seperti di atas, kompilasi, coba jalankan, dan pastikan tidak ada error.

## 4. Kesalahan Umum Terkait Inheritance

Coba ubahlah kelas Tester Anda menjadi kode sederhana ini dan jalankan:

```
public class Tester {
    public static void main(String[] args) {
        Integer i1 = 13;
        Number n1 = i1;
        System.out.println(n1);
    }
}
```

---

<sup>5</sup> Comment out dengan cara menambahkan sintaks komentar “//” di depan baris tersebut. Akibatnya, baris tersebut akan dilompati compiler, namun bisa Anda kembalikan dengan mudah saat nanti dibutuhkan.

8. Mengapa sebuah variabel dengan tipe data Number bisa diisi variabel dengan tipe data Integer? Prinsip apa yang dipakai di sini?

Sekarang, cobalah bungkus kedua variabel tersebut dengan Kardus, seperti ini:

```
public class Tester {  
    public static void main(String[] args) {  
        Kardus<Integer> i1 = new Kardus<>(13);  
        Kardus<Number> n1 = i1;  
        System.out.println(n1);  
    }  
}
```

Lakukan kompilasi, dan perhatikan dan coba pahami pesan kesalahannya.

Hal ini terjadi karena, walaupun Integer merupakan turunan dari Number, tidak berarti Kardus<Integer> merupakan turunan dari Kardus<Number>. Baik Kardus<Number> maupun Kardus<Integer>, keduanya merupakan turunan dari kelas Object.

Gabungkan Kardus.java, Pasangan.java, PasanganKonkrit.java, Tester.java, dan PT0701xxyyy.txt dalam 1 file zip lalu kumpulkan dengan nama PT0701xxyyy.zip.